Programa:	Clip Plus 3.3
Protección:	ASProtect 1.23 RC4 + Trial 30 días
Objetivos:	Desempakar y evitar que expire
URL descarga:	http://www.hotdownloads.com
Dificultad:	No es difícil
Herramientas:	OllyDbg v1.10 +PEiD 0.94 + ResHacker 3.4
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinoS] Fecha: 13-11-05

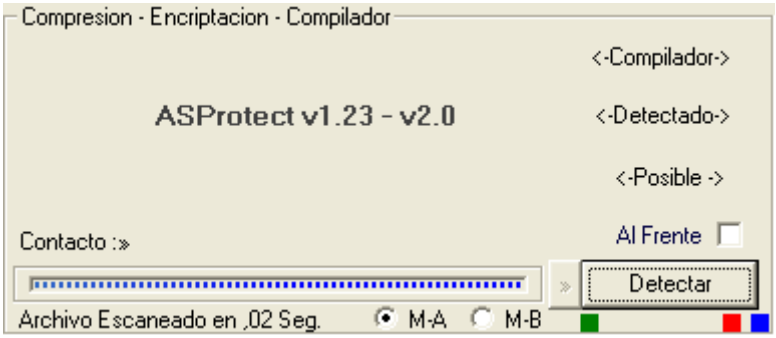
Hola a todo CracksLatinoS una vez más..!!

Hoy veremos como desempakar un ASProtect de los antiguos, hoy día está de moda empaketarlos con la versión 2.0x [Registered] cuyo fuerte está en la redirección de IAT, estas versiones anteriores se protegen mucho también en la IAT y muy probablemente en algunos antidumps, varias veces lo he visto y no corren, además de esto suelen tener stolen bytes al inicio del código. He de decir que todavía no lo he desempakado y no sé si lo conseguiré, si estás leyendo esto seguramente lo habré conseguido sino este tute no vería la luz al no estar terminado, así que nos vamos a trabajar, jeje…

Ejecutamos el nuevo PEiD 0.94 gentileza de k0rt ¿? Aunque fue Red quien lo envió a la lista si no recuerdo mal, cargamos el programa víctima y nos dice esto:g

ASProtect 1.23 RC4 Registered -> Alexey Solodovnikov

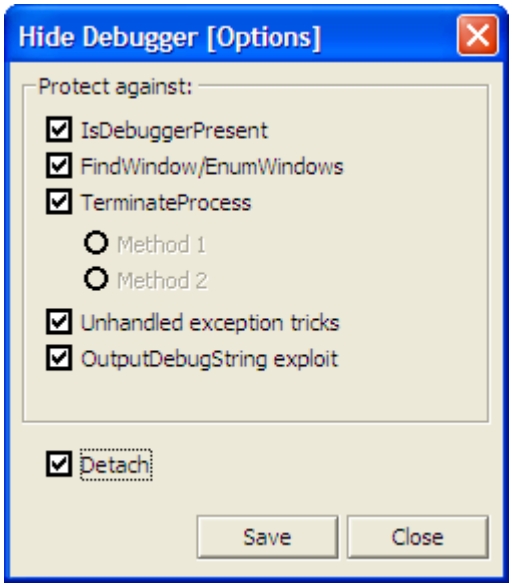
Lo que hemos dicho, la versión Registrada del 1.23 RC4 veremos que nos trae, pero antes comocuriosidad le echaremos un vistazo con RDG packer a ver que opina:



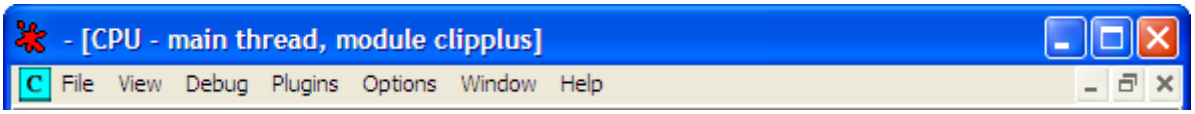
Lo mismo un 1.23 lo de 2.0 no se porque lo pone, quizá porque no puede concretar bien la versión como sí hace PEiD, una vez analizado el archivo lo llevamos al Olly:

```
00401000  68 01007100   PUSH clipplus.00710001
00401005  E8 01000000   CALL clipplus.0040100B
0040100A  C3            RETN
0040100B  C3            RETN
```

Ahí estamos en el EntryPoint del programa, para que corra bien deberemos tener oculto el debugger, yo utilizo el plugin HideDebugger con estas opciones:



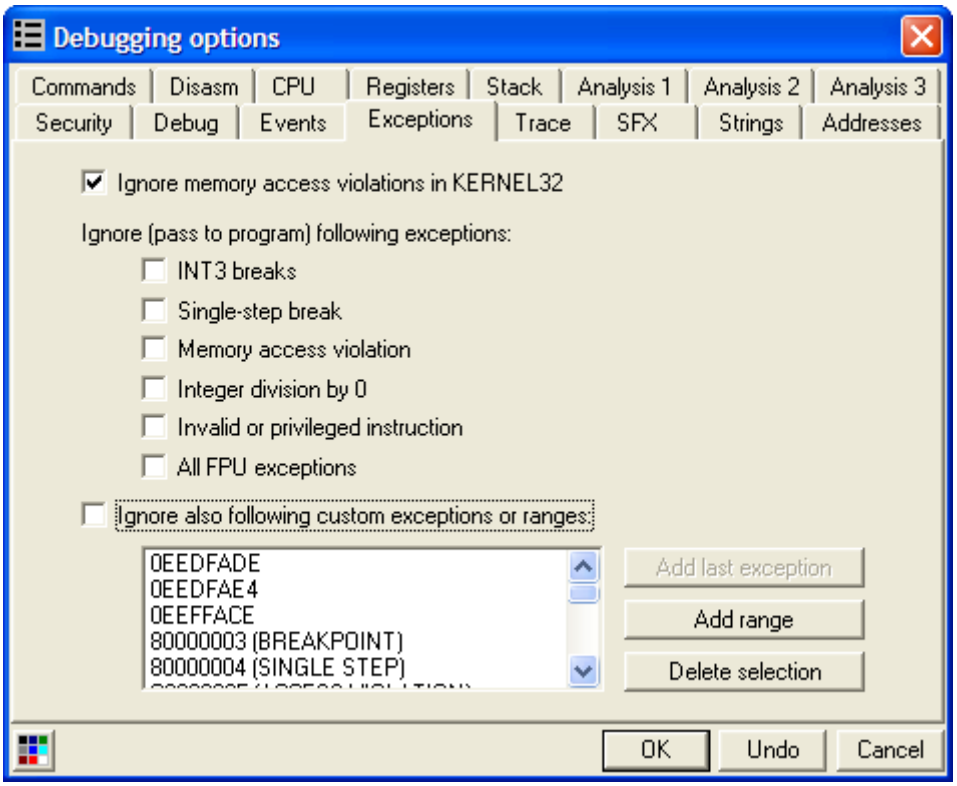
Y también tengo el de IsDebuggerPresent, esto me evita la detección del Olly y algunos truquitos más de los que se encarga el propio plugin activando las casillas "Unhandled exception tricks" para las excepciones, el "OututDebugString exploit" bien conocida por todos sobretodo por los Armadillos que la usan bastante, "Terminate Process" que no se bien bien para que se utiliza y el "FindWindow/EnumWindow" que nos oculta la ventana del Olly, este no me hace falta porque tengo el Olly parcheado:



de todos modos la opción "FindWindow/EnumWindow" nos deja el Olly así ;-)

Además de estos dos plugins tengo también el OllyScript 0.92 y como no uno de los más importantes: el CommandBar v3.00.108 y por último el Ollydump que utilizo bastante, prácticamente suelo dumpear siempre con él salvo raras excepciones.

La ventana de excepciones la tengo de esta manera:

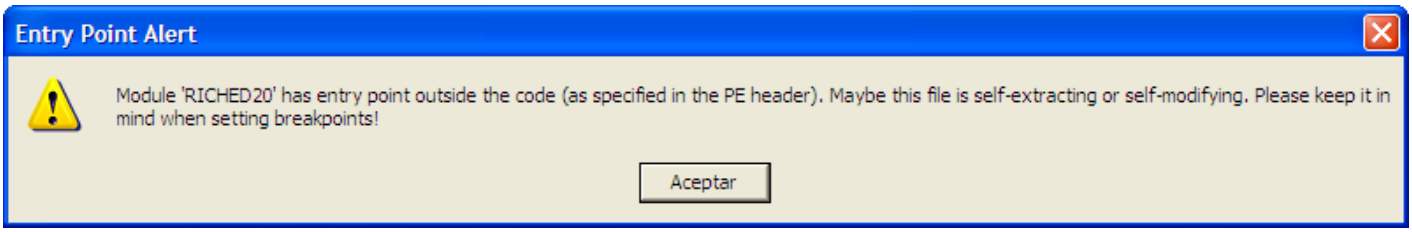


Esta es la configuración con la que trabajaremos en un principio, si hiciera falta algo más lo iríamos comentando sobre la marcha, ahora vayamos al tajo xDD

Lo primero que haremos como siempre será localizar el OEP con el método de las excepciones, las pasamos todas excepto esta que es la última en mi máquina:

```
00D93D03   3100            XOR DWORD PTR DS:[EAX],EAX
00D93D05   64:8F05 000000  POP DWORD PTR FS:[0]
00D93D0C   58              POP EAX
00D93D0D   833D BC7ED900   CMP DWORD PTR DS:[D97EBC],0
00D93D14 v 74 14           JE SHORT 00D93D2A
00D93D16   6A 0C           PUSH 0C
```

después de esta nos aparece este mensaje:



Nos dice que el módulo RICHED20 tiene el EP fuera de la sección de código, primer síntoma de que puede estar comprimido, se refiere a la librería riched20.dll que tiene en su misma carpeta de instalación.

Reiniciamos Olly con Ctrl+F2 y llegamos a la última excepción, vamos al Memory map con Alt+M y ponemos un BP en la primera sección (la de código):

```
Address  Size     Owner    Section  Contains   Type Access Initial Mapped as
00380000 00001000                              Priv RWE    RWE
00390000 00001000                              Priv RWE    RWE
003A0000 00001000                              Priv RWE    RWE
003B0000 00005000                              Priv RW     RW
003C0000 00003000                              Map  R      R       \Device\HarddiskVolume1\W
003D0000 00008000                              Priv RW     RW
003E0000 00001000                              Priv RW     RW
003F0000 00001000                              Priv RW     RW
00400000 00001000 clipplus         PE header   Imag R      RWE
00401000 00218000 clipplus         code
00619000 0001E000 clipplus
00637000 00001000 clipplus
00638000 00001000 clipplus
00639000 00004000 clipplus
0063D000 0004E000 clipplus         export
0068B000 00062000 clipplus .rsrc   resour
006ED000 00023000 clipplus
00710000 00017000 clipplus .data   data,i
00727000 00001000 clipplus .adata
00730000 00001000 ole32            PE hea
00731000 00104000 ole32    .text   code,i
00835000 00006000 ole32    .orpc   code
0083B000 00007000 ole32    .data   data
00842000 00002000 ole32    .rsrc   resour
00844000 0000D000 ole32    .reloc  reloca
00860000 0000A000
00920000 00002000
```

```
Actualize
View in Disassembler                Enter
Dump in CPU
Dump
Search                              Ctrl+B
Set break-on-access                 F2
Set memory breakpoint on access
Set memory breakpoint on write
```

La superamos ahora sí con Shift+F9 y estamos en el OEP ;-)

```
00401000  EB 10          JMP SHORT clipplus.00401012
00401002  66:623A        BOUND DI,DWORD PTR DS:[EDX]
00401005  43             INC EBX
00401006  2B2B           SUB EBP,DWORD PTR DS:[EBX]
00401008  48             DEC EAX
00401009  4F             DEC EDI
0040100A  4F             DEC EDI
0040100B  4B             DEC EBX
0040100C  90             NOP
0040100D  E9 A4996100    JMP 00A1A9B6
00401012  A1 97996100    MOV EAX,DWORD PTR DS:[619997]
00401017  C1E0 02        SHL EAX,2
```

OEP==401000

Anotamos este dato:

OEP: 401000

Por la dirección de inicio y por la estructura del código podemos casi asegurar que está compilado en MASM/TASM, pues casi siempre empiezan en :401000 y además lo hacen con un JMP, tampoco vemos que tenga stolen bytes, ya que estamos aquí vamos a delimitar la IAT, buscamos un puntero a la misma, entrando en el primer CALL de la dirección :401022 pulsando Enter o con la opción "Follow" encontramos la tabla de saltos:

```
006176F2  FF25 DC946300  JMP DWORD PTR DS:[6394DC]
006176F8  FF25 E0946300  JMP DWORD PTR DS:[6394E0]
006176FE  FF25 E4946300  JMP DWORD PTR DS:[6394E4]
00617704  FF25 E8946300  JMP DWORD PTR DS:[6394E8]
0061770A  FF25 EC946300  JMP DWORD PTR DS:[6394EC]
00617710  FF25 F0946300  JMP DWORD PTR DS:[6394F0]
00617716  FF25 F4946300  JMP DWORD PTR DS:[6394F4]
0061771C  FF25 F8946300  JMP DWORD PTR DS:[6394F8]
00617722  FF25 FC946300  JMP DWORD PTR DS:[6394FC]
00617728  FF25 00956300  JMP DWORD PTR DS:[639500]
0061772E  FF25 04956300  JMP DWORD PTR DS:[639504]
00617734  FF25 08956300  JMP DWORD PTR DS:[639508]
```

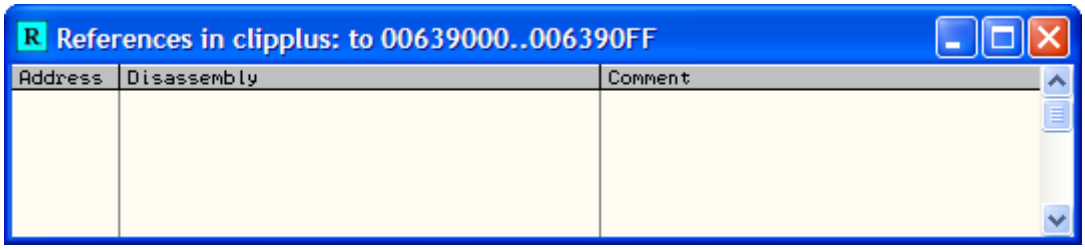
Como vemos está completamente destrozada, apenas se ha salvado algún puntero, estos son los del kernel y suelen llevarse la peor parte en este packer ya que los redirecciona casi todos, picamos con el botón derecho sobre un salto cualquiera de estos y elegimos "Follow in dump->Memory Address" en la ventana de dump nos mostrará la entrada, sólo tenemos que subir hasta encontrar el inicio de la IAT:

```
Address   Hex dump                                          ASCII
0063913C  3E 93 8E 49 78 5D 6D 7D DD A1 55 F8 8C 1A 95 78  >ôÄIx]m}¦¡Uº¶↑òx
0063914C  FB C9 3B DE 8D E1 67 94 B4 C3 63 91 D1 6E A0 14  ¹╓;▌ìßgö┤├cæÐnà¶
0063915C  A5 7C 32 5D 6C 73 71 3F 04 60 B3 06 71 FB 44 55  ñ¦2]lsq?◆`≥♠q¹DU
0063916C  CE D8 01 4F 8B 50 58 8B 4C 25 8A F0 E0 78 2B 1D  ╬╪☺O▌PX▌L%è-ðx+↔
0063917C  91 69 6A 56 4B 42 B1 6C 82 0C 9B 29 D7 DF 60 2C  æijVKB▒lé.¢)╫▀`,
0063918C  B8 5D 2C 42 A3 13 08 A6 C3 C6 40 B7 FC E5 7C 0A  ®],Bú‼◘ª╠╞@Ã·ü|.
0063919C  B4 90 E2 ED D8 17 DA 77 D6 27 DA 77 E2 68 DB 77  ┤É▒Ýï$↨rwï'rwÔh▌w
006391AC  D4 65 DB 77 8B 6F DB 77 68 6A DB 77 6D 69 DB 77  Ôe▌wï▌wh▌wmi▌w
006391BC  D9 23 DA 77 9A 22 DA 77 F4 6C DB 77 5B 66 DB 77  ┘#rwü"rw¶l▌w[f▌w
006391CC  10 24 DA 77 8B 1B DA 77 B1 63 DB 77 F6 8D FD DA  ►$rwï←rw▒c▌w÷ï²r
006391DC  C1 67 A0 B6 F2 6E E0 56 16 4E 91 15 F3 AD 4C 6F  ┴gáÃ=nóV▬Næ§%↓Lo
```

En este caso no se ve muy claro el inicio, para no equivocarnos podemos utilizar el comando "Find References" que se invoca con Ctrl+R y nos ayuda, un ejemplo: seleccionamos una zona en la que sospechamos puede haber algún puntero:

Address	Hex dump	ASCII
00639000	73 B3 99 BE C9 4F 4E 66 0B FC FA B8 D8 75 E9 33	s|Ö¥╔ONf♂ª·Ø¡uÚ3
00639010	BA 94 4B F6 04 0E A8 AD AA BF 7D 3E 51 27 6E 7E	‖öK÷♦Ñ¿↓¬ }>Q'n~
00639020	1E AF B8 2E 1E D5 DF E0 42 60 A4 B1 50 20 D3 D7	▲»Ø.▲ÕßàB`¤±P ╙╫
00639030	3C 03 7E 7D CD 68 94 74 8F F8 09 A0 51 79 EF 33	<♥~}═hötÅ°.áQy'3
00639040	D9 5E 3F 22 81 12 1C 5C CF 28 E3 B2 5F 0D 63 7E	┘^?"ü↕∟\╧(ã▒_.c~
00639050	98 CD D6 6B 09 16 AD F9 70 84 3E 9D C0 E7 C9 90	ÿ═ïk.▬↓¨pä>Ø└╠╔É
00639060	A1 0C 85 B0 4D 25 95 13 C2 0A 20 14 A7 B9 E9 16	í.à▒M%ò‼┬. ¶º┐ú▬
00639070	56 F2 25 3B F8 CA 60 BA A7 89 BD B2 E6 42 E4 86	V≡%;°╩`‖º§ë¢▒µBäå
00639080	05 E7 57 39 2F D9 12 47 42 1C AF F3 9B C5 68 10	♣╠W9/┘↕GB∟»╝╬►h►
00639090	90 52 B1 AB 37 E0 4E 3C A4 90 1A 1D E1 79 DD 89	ÉR▒½7ÓN<╟É►↔ßy¦ë
006390A0	28 05 F3 59 2C 9A 8D 3E 84 DB E0 33 82 F5 98 61	(♣≡Y,ÜÍ>ä▄Ó3é§ÿa
006390B0	F3 B1 34 BE AE 57 4A 03 E8 89 D9 E2 A3 A5 08 90	╠▒4¥«WJ♥Ê┘ôúÑ▐É
006390C0	C3 59 10 F7 97 76 35 40 D7 2A F5 77 77 34 E5 85	├Y►╤ùv5@ï*§ww4Ôå
006390D0	C1 BA DC BA A1 CD 5F 9B 0C C4 78 C8 EF 00 67 19	┴‖▄‖ ï=_¢.─x╚'.g↓
006390E0	23 C2 D4 3C 1D 21 6C 9A 9D 49 22 2B 6A 8B 6E 79	#╥Ê<↔!lÜØI"+jïny
006390F0	D1 FA 4C 2A A2 8C C6 93 B8 FA 64 5C 62 E8 B6 21	Ð·L*óïäõ@·d\bÊÂ!
00639100	25 FD DC 93 BB F3 C9 9E 49 E2 8F 7A 1F 2E 08 C0	%²▄ôº¾╔×IôÂz▼.▐└
00639110	8C E1 98 DB 9A 7A F3 7F AD 44 02 EC 66 E2 63 2F	îßÿ▐Üz¾♠↓D☻ÿfôc/
00639120	3F AE 39 AC C4 E9 18 9F 64 03 5A 57 2B 70 38 5E	?«9¼─ú↑fd♥ZW+p8^
00639130	EE C6 4D E2 C5 27 8A F5 BE 1F A7 8C 3E 93 8E 49	¯ãMÓ┼'è§¥▼º§î>ôÄI
00639140	78 5D 6D 7D DD A1 55 F8 8C 1A 95 78 FB C9 3B DE	x]m}¦¡U°ï→ôx'╓;ï
00639150	8D E1 67 94 B4 C3 63 91 D1 6E A0 14 A5 7C 32 5D	ìßgöÁ├cæÐná¶Ñ¦2]
00639160	6C 73 71 3F 04 60 B3 06 71 FB 44 55 CE D8 01 4F	lsq?♦`═♠q'DU╬ïØ☺O
00639170	8B 50 58 8B 4C 25 8A F0 E0 78 2B 1D 91 69 6A 56	ïPXïL%è-ðx+↔æ¡jV

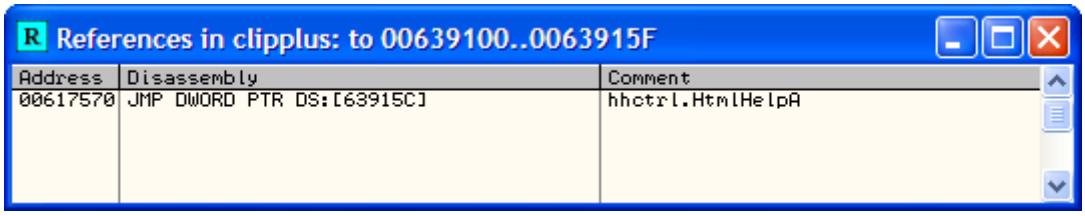
pulsamos Ctrl+R y nos aparece esto:



En esa zona no hay ningún puntero, si hacemos lo mismo un poco más abajo:

Address	Hex dump	ASCII
00639100	25 FD DC 93 BB F3 C9 9E 49 E2 8F 7A 1F 2E 08 C0	%²▄ôº¾╔×IôÂz▼.▐└
00639110	8C E1 98 DB 9A 7A F3 7F AD 44 02 EC 66 E2 63 2F	îßÿ▐Üz¾♠↓D☻ÿfôc/
00639120	3F AE 39 AC C4 E9 18 9F 64 03 5A 57 2B 70 38 5E	?«9¼─ú↑fd♥ZW+p8^
00639130	EE C6 4D E2 C5 27 8A F5 BE 1F A7 8C 3E 93 8E 49	¯ãMÓ┼'è§¥▼º§î>ôÄI
00639140	78 5D 6D 7D DD A1 55 F8 8C 1A 95 78 FB C9 3B DE	x]m}¦¡U°ï→ôx'╓;ï
00639150	8D E1 67 94 B4 C3 63 91 D1 6E A0 14 A5 7C 32 5D	ìßgöÁ├cæÐná¶Ñ¦2]
00639160	6C 73 71 3F 04 60 B3 06 71 FB 44 55 CE D8 01 4F	lsq?♦`═♠q'DU╬ïØ☺O
00639170	8B 50 58 8B 4C 25 8A F0 E0 78 2B 1D 91 69 6A 56	ïPXïL%è-ðx+↔æ¡jV

y buscamos referencias ahora sí que encontramos una:



Entonces sabemos que entre todo este bloque seleccionado hay un puntero de la IAT, sólo hay que buscarlo, en este caso es el del inicio de tabla, lo anotamos:

iat_inicio: 63915C

Hacemos lo mismo con el final, si tuviéramos problemas en encontrarlo siempre podemos utilizar este método explicado ya que no suele fallar, a no ser en casos como Armadillo que a veces no muestra las referencias aunque realmente haya, el final es estse:

Address	Hex dump	ASCII
0063A400	E2 19 0F 77 EC 14 0F 77 B8 AB 0F 77 44 16 0F 77	Ô↓*wÿ¶*w@½*wD▬*w
0063A410	5D 86 AF FC 43 A1 08 C6 13 24 F9 01 88 07 87 5A	]å»ⁿÇí▐á‼$¨Øè·çZ
0063A420	FD B9 09 14 14 E9 CC 74 18 F7 CC 74 47 EA CC 74	²╢.¶¶Ù╟t↑┤╟tGÙ╟t
0063A430	23 16 6E C5 00 02 7A 36 6B 63 88 FB 2D D7 EB 82	#▬n┼.☻z6kcê¹-╫Ùé
0063A440	22 3A 92 9F 11 25 98 14 8E 6A 8F AA 1E FE EF 25	":Æƒ◄%ÿ¶ÄjÅ¬▲■'%

ocurre lomismo que con el inicio que no se ve muy bien la zona porque está llena de datos, además la IAT está muy agujereada tiene mucha basura entremedias que habrá que limpiar con un "Cut Thunks" después pero eso viene luego, de momento anotamos el final de tabla que hemos obtenido:

iat_final: 63A43C

Con estos dos datos obtenemos su longitud fácilmente:

iat_largo: 12E0

No es que sea una Iat muy grande sino que tiene mucha basura por eso nos sale 12E0h de largo que es bastante (las hay mayores por cierto) lo siguiente será encontrar el call semi_mágico que se le llama del que nos valdremos para reparar gran parte de la IAT, jeje.. miremos los primeros saltos de la JUMp_TABLE:

```
006175AE - FF25 C4916300  JMP DWORD PTR DS:[6391C4]   ADVAPI32.RegQueryInfoKeyA
006175B4 - FF25 C8916300  JMP DWORD PTR DS:[6391C8]   ADVAPI32.RegQueryValueA
006175BA - FF25 CC916300  JMP DWORD PTR DS:[6391CC]   ADVAPI32.RegQueryValueExA
006175C0 - FF25 D0916300  JMP DWORD PTR DS:[6391D0]   ADVAPI32.RegQueryValueExW
006175C6 - FF25 D4916300  JMP DWORD PTR DS:[6391D4]   ADVAPI32.RegSetValueExA
006175CC - FF25 18946300  JMP DWORD PTR DS:[639418]   kernel32.CloseHandle
006175D2 - FF25 1C946300  JMP DWORD PTR DS:[63941C]
006175D8 - FF25 20946300  JMP DWORD PTR DS:[639420]
006175DE - FF25 24946300  JMP DWORD PTR DS:[639424]
006175E4 - FF25 28946300  JMP DWORD PTR DS:[639428]   kernel32.CreateFileA
006175EA - FF25 2C946300  JMP DWORD PTR DS:[63942C]
```

hacemos un "Follow in dump->Memory Address" en el primero que está redirigido [63941C] cuyo valor final es DA8DC0 como se ve en la siguiente imagen:

```
Address   Hex dump                                          ASCII
0063941C  C0 8D DA 00 9C 04 DA 00 AC 04 DA 00 76 B4 E5 77  └ìr.£♦r.¼♦r.v┤ðw
0063942C  BC 04 DA 00 F8 02 DA 00 CC 04 DA 00 F0 04 DA 00  ╝♦r.°☻r.╠♦r.-♦r.
0063943C  FC 04 DA 00 10 05 DA 00 20 05 DA 00 30 05 DA 00  ³♦r.►♣r. ♣r.0♣r.
0063944C  6C 03 DA 00 FD 98 E5 77 3C 05 DA 00 50 05 DA 00  l♥r.²ÿ˜w<♣r.P♣r.
0063945C  6C 05 DA 00 7C 05 DA 00 8C 05 DA 00 A0 05 DA 00  l♣r.|♣r.î♣r.á♣r.
```

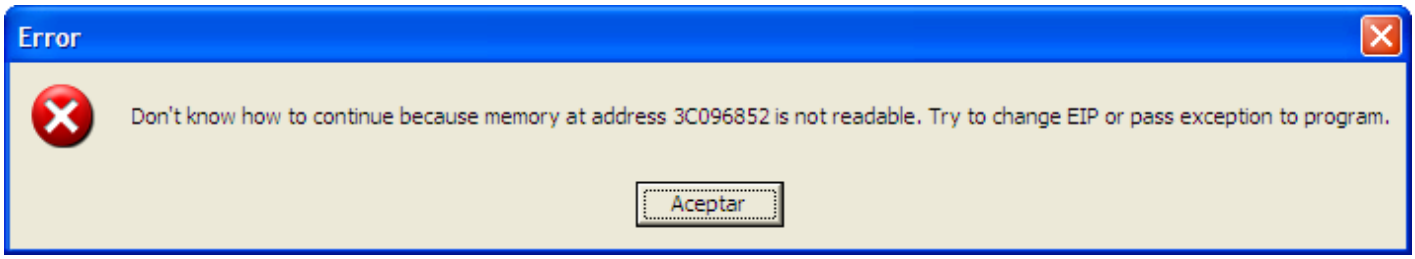
Estos bytes nos ayudaran a descubrir la zona donde rellena la IAT, lo que debemos hacer es lo siguiente: Reiniciamos Olly, estando en el EP hacemos un GO a la dirección del puntero MALO, y allí le ponemos un "BMP on Write" no pongo imagen porque no tiene sentido es algo donde no se puede perder nadie, luago vamos pulsando F9 hasta llegar al punto donde guarda los valores finales que ya conocemos, para ir más rápido podemos deshabilitar las excepciones de momento luego ya las devolveremos a su estado inicial:

```
00D935CB   50            PUSH EAX
00D935CC   E8 43FCFFFF   CALL 00D93214
00D935D1   E8 7EFEFFFF   CALL 00D93454                ;este es el magico, jeje...
00D935D6   8B17          MOV EDX,DWORD PTR DS:[EDI]
00D935D8   8902          MOV DWORD PTR DS:[EDX],EAX   ;aqui guarda..!!
00D935DA v EB 7E         JMP SHORT 00D9365A
```

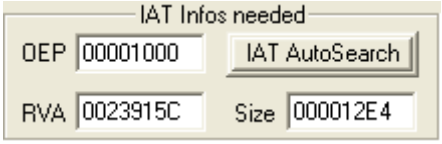
No tardamos mucho en llegar aquí xDD una zona muy bien conocida por todos nosotros, esos dos CALL's seguidos de esta versión, ahora podemos tracear el bucle en el que estamos para ver como trabaja, esto es totalmente opcional ya que podemos darle caña con los datos que tenemos, si queremos meterle mano ya podemos, anotamos la dirección del call_mágico :D935D1, para legar a él primero tenemos que dejar que se cree el bloque de memoria donde irá el CALL ya que lo debe pedir primero con VitualAlloc, la solución más fácil es reiniciar Olly habilitar las excepciones para que vaya parando, cuando pare en la primera de ellas volver a deshabilitarlas y hacer un GO a la direcicón del CALL mágico y nopearlo:

```
00D935C7   56            PUSH ESI
00D935C8   8B45 0C       MOV EAX,DWORD PTR SS:[EBP+C]
00D935CB   50            PUSH EAX
00D935CC   E8 43FCFFFF   CALL 00D93214
00D935D1   90            NOP
00D935D2   90            NOP
00D935D3   90            NOP
00D935D4   90            NOP
00D935D5   90            NOP
00D935D6   8B17          MOV EDX,DWORD PTR DS:[EDI]
00D935D8   8902          MOV DWORD PTR DS:[EDX],EAX
00D935DA v EB 7E         JMP SHORT 00D9365A
```

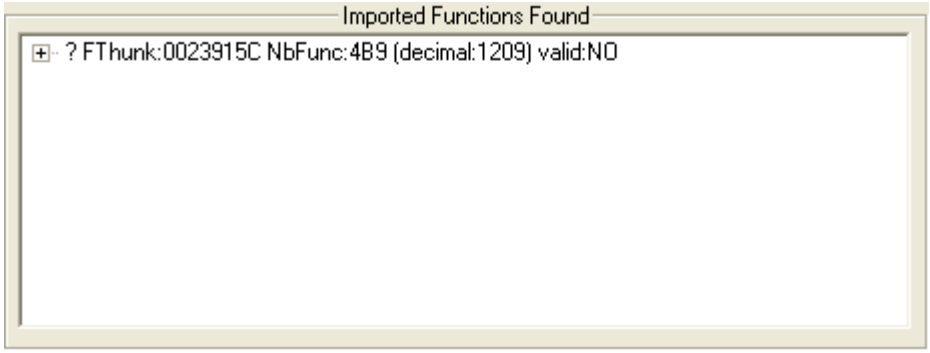
Antes de continuar debemos poner un BPM en la última entrada de la IAT [63A43C] y pulsar F9 para que corra, al final debería parar por el BPM pero nos sale esto:



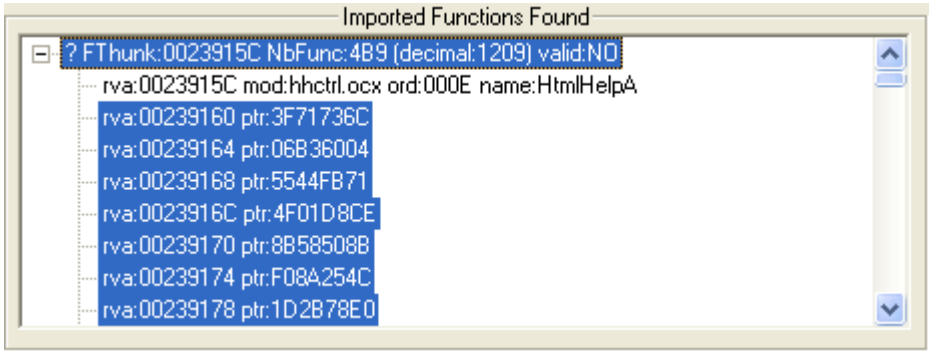
En un principio pensé que había hecho algo mál ¿? pero si en este momento abrimos el Import Reconstructor y le damos los datos correctos de la IAT que son estos:



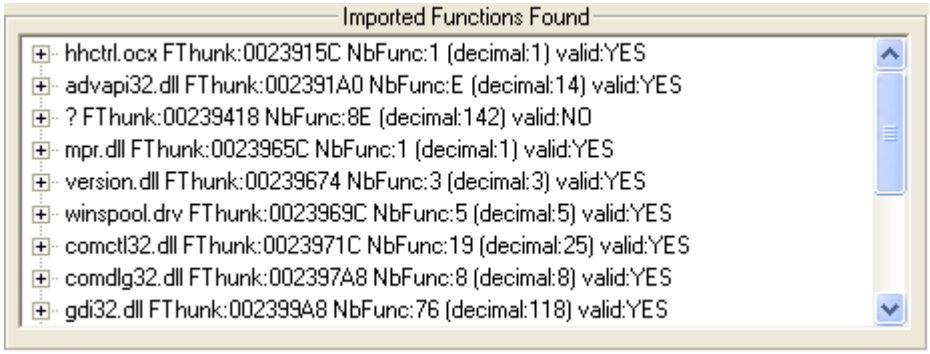
no creo que merezcan más explicación verdad? Picamos "Get Imports" y sólo aparece un thunk ¿se dice así xDD? Y dice Valid:NO, vemos esto:



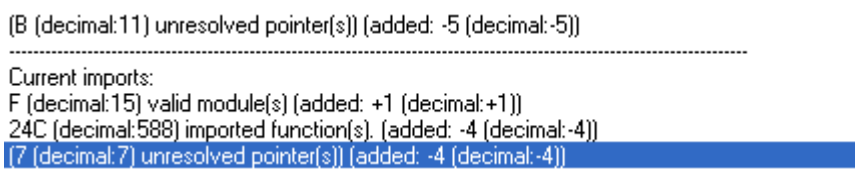
Si pulsamos "Show Invalid2 nos muestra las entradas supuestamente MALAS:



Vemos que son valores totalmente inofensivos, podemos ignorarlos, recordad que ya dijimos que esta era una IAT muy agujereada, pero antes de ir a lo loco para no meter la pata lo que haremos es seleccionar las entradas MALAS que estén consecutivas, picamos en la primera con el botón izquierdo, vamos a la última del bloque y con ela tecla Shift pulsada y sin soltarla picamos en la última, quedará todo el bloque seleccionado y podemos hacer "Cut thnunks" de ese bloque únicamente, así recuperamos gran parte de la tabla, quedarán 7 por resolver:



Si miramos el Log vemos que son sólo 7 las que nos quedaron por resolver:



A todo esto he de decir que el error anterior es debido a que calculé mal el final de tabla puse 63A43C cuando es 63A42C de ahí deriva el error que vimos antes pero no hay problema, abrimos otro Olly y llegamos al OEP normalmente con el método de las

excepciones y lo dejamos parado, de mientras en el Olly del error vamos reparando las siete entradas del kernel que quedaron MALAS, son estas:

IAT_rva	Apunta_a	API_correcta
23947C	D9158C	FreeResource
239488	D91574	GetCommandLineA
239490	D9155C	GetCurrentProcessId
2394DC	D91500	GetModuleHandleA
2394E8	D910AC	GetProcAddress
23951C	D91528	GetVersion
239590	D91564	LockResource

Las iremos viendo todas para los que empiezan las sepan reconocer:

Entrada1:
IAT_rva: 23947C
Apunta a: D9158C

DISASSEMBLER / HEX VIEWER

```
00D9158C    push ebp
00D9158D    mov ebp,esp
00D9158F    mov eax,[D97CF0]    // DWORD value: 00151EE0
00D91595    pop ebp
00D91596    retn 4
```

API_correcta: FreeResource

Entrada2:
IAT_rva: 239488
Apunta a: D91574

DISASSEMBLER / HEX VIEWER

```
00D91574    push 0
00D91576    call 00D854AC    // = kernel32.dll/016F/GetModuleHandleA
00D9157B    push dword ptr [D97CE0]    // DWORD value: 0A280105
00D91581    pop eax
00D91582    mov eax,[D97CF0]    // DWORD value: 00151EE0
00D91588    retn
```

API_correcta: GetCommandLineA

Entrada3:
IAT_rva: 239490
Apunta a: D9155C

DISASSEMBLER / HEX VIEWER

```
00D9155C    mov eax,[D97CEC]    // DWORD value: 00000FD8
00D91561    retn
```

API_correcta: GetCurrentProcessId

Entrada4:
IAT_rva: 2394DC
Apunta a: D91500

DISASSEMBLER / HEX VIEWER

```
00D91500    push ebp
00D91501    mov ebp,esp
00D91503    mov eax,[ebp+8]
00D91506    test eax,eax
00D91508    jnz short 00D9151D
00D9150A    cmp dword ptr [D97970],400000
00D91514    jnz short 00D9151D
00D91516    mov eax,[D97970]     // DWORD value: 00400000
00D9151B    jmp short 00D91523
00D9151D    push eax
00D9151E    call 00D854AC     // = kernel32.dll/016F/GetModuleHandleA
00D91523    pop ebp
00D91524    retn 4
```

API_correcta: GetModuleHandleA

Entrada5:
IAT_rva: 2394E8
Apunta a: D910AC

DISASSEMBLER / HEX VIEWER

```
00D910AC    push ebp
00D910AD    mov ebp,esp
00D910AF    mov edx,[ebp+C]
00D910B2    mov eax,[ebp+8]
00D910B5    mov ecx,[D96474]     // DWORD value: 00D96314
00D910BB    mov ecx,[ecx]
00D910BD    cmp ecx,eax
00D910BF    jnz short 00D910CA
00D910C1    mov eax,[edx*4+D96350]     // DWORD value: 00000000
00D910C8    jmp short 00D910D1
00D910CA    push edx
00D910CB    push eax
00D910CC    call 00D854B4     // = kernel32.dll/0191/GetProcAddress
00D910D1    pop ebp
00D910D2    retn 8
```

API_correcta: GetProcAddress

Entrada6:
IAT_rva: 23951C
Apunta a: D91528

DISASSEMBLER / HEX VIEWER

```
00D91528    push dword ptr [D97CE0]     // DWORD value: 0A280105
00D9152E    pop eax
00D9152F    retn
```

API_correcta: GetVersion

Entrada7:
IAT_rva: 239590
Apunta a: D91564

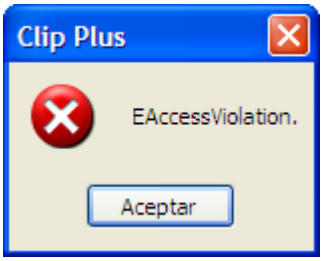
```
DISASSEMBLER / HEX VIEWER

00D91564    push ebp
00D91565    mov ebp,esp
00D91567    mov eax,[D97CF0]     // DWORD value: 00151EE0
00D9156D    mov eax,[ebp+8]
00D91570    pop ebp
00D91571    retn 4
```

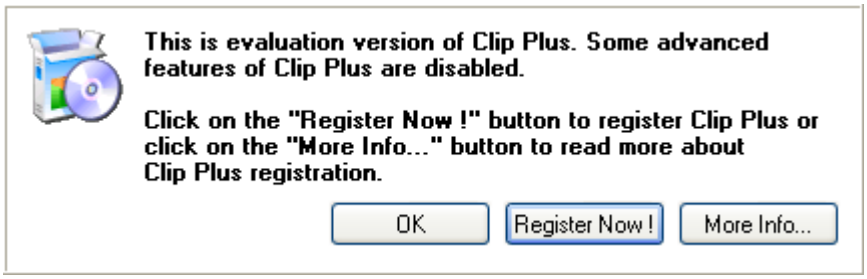
API_correcta: LockResource

Estas son las siete entradas que quedaron por resolver ;-)) Ahora que las tenemos salvamos la IAT al disco, esto lo hacemos desde el ImpRec pulsando "Save Tree" y la llamamos tute_iat.txt, antes os dije que tuviésemos otro Olly con el programa detenido en el OEP verdad? Pues ahora lo necesitamos, debido al error al calcular la IAT nos petó, no hubiera hecho falta esto si lo hubiésemos hecho bien pero como estoy haciendo el tute sobre la marcha ocurren estas cosas pero no pasa nada lo haremos igual, en el otro Olly dumpeamos con las opciones de siempre y lo salvamos como tute.exe

Ahora que tenemos el dumpeado desde el ImpRec pulsamos "Fix Dump" y elegimos tute.exe, nos creará tute_.exe como todos sabemos y es el que deberemos probar, hacemos doble clic sobre él y primer petardazo ¿?



Nada, cerramos los dos Ollys y en otro cargamos tute_.exe vamos a investigar el error a ver como lo podemos evitar, es increíble ahora parece que funciona, sale la nag…



Pulsamos OK y el programa carga perfecto, aprovechamos que estábamos buscando el error localizamos el call_nag, es este:

```
00401353  .  BA 02000000  MOV EDX,2
00401358  .  E8 0FEF1E00  CALL tute_.005F026C
0040135D  .  8B0D CC0E630 MOV ECX,DWORD PTR DS:[630ECC]      tute_.00632D60
00401363  .  8B01         MOV EAX,DWORD PTR DS:[ECX]
00401365  .  8B0D 480C630 MOV ECX,DWORD PTR DS:[630C48]      tute_.00631524
0040136B  .  8B15 1C21620 MOV EDX,DWORD PTR DS:[62211C]      tute_.00622168
00401371  .  E8 F69D1800  CALL tute_.0058B16C                ;call_nag...
00401376  .  A1 CC0E6300  MOV EAX,DWORD PTR DS:[630ECC]
0040137B  .  8B00         MOV EAX,DWORD PTR DS:[EAX]
0040137D  .  8B0D 600C630 MOV ECX,DWORD PTR DS:[630C60]      tute_._ClbViewForm
00401383  .  8B15 C853620 MOV EDX,DWORD PTR DS:[6253C8]      tute_.00625414
00401389  .  E8 DE9D1800  CALL tute_.0058B16C
0040138E  .  A1 CC0E6300  MOV EAX,DWORD PTR DS:[630ECC]
```

Pero no hay ningún salto bueno cerca, dejamos puesto el BP y reiniciamos, cuando Olly pare entramos con F7, de momento traceamos con F7 para entrar en algunos CALL que veamos sospechosos bien porque tengan un salto condicional antes o después o algo que nos llame la atención, al final localizamos el subcall que se encarga de mostrar la nag en pantalla, lo mejor de todo es que ya se empiezan a ver saltos que podrían irnos bien, veamos la zona más tranquilamente:

```
00406334  .   E8 53780000   CALL tute_.0040DB8C
00406339  .   8B55 FC       MOV EDX,DWORD PTR SS:[EBP-4]
0040633C  .   80BA 2D07000( CMP BYTE PTR DS:[EDX+72D],0          ;bonita comparacion...
00406343  .v  0F85 8000000( JNZ tute_.004063C9                   ;ojo a este salto...
00406349  .   8B4D FC       MOV ECX,DWORD PTR SS:[EBP-4]
0040634C  .   B2 01         MOV DL,1
0040634E  .   A1 A0476200   MOV EAX,DWORD PTR DS:[6247A0]
00406353  .   E8 64600100   CALL tute_.0041C3BC
00406358  .   8BD8          MOV EBX,EAX
0040635A  .   A1 540C6300   MOV EAX,DWORD PTR DS:[630C54]
0040635F  .   8918          MOV DWORD PTR DS:[EAX],EBX
00406361  .   8BC3          MOV EAX,EBX
00406363  .   8B10          MOV EDX,DWORD PTR DS:[EAX]
00406365  .   FF92 D800000( CALL DWORD PTR DS:[EDX+D8]           ;call_nag...
0040636B  .   8B0D 540C630( MOV ECX,DWORD PTR DS:[630C54]        tute_._RegNoteForm
00406371  .   8B19          MOV EBX,DWORD PTR DS:[ECX]
00406373  .   895D C0       MOV DWORD PTR SS:[EBP-40],EBX
00406376  .   85DB          TEST EBX,EBX
00406378  .v  74 24         JE SHORT tute_.0040639E
0040637A  .   8B03          MOV EAX,DWORD PTR DS:[EBX]
0040637C  .   8945 C4       MOV DWORD PTR SS:[EBP-3C],EAX
```

La nag aparece en :406365 y hay un salto que evita pasar por allí justo antes en :406343, podríamos poner un BP en dicho salto a ver si podemos evitarla limpiamente, si pausamos Olly cuando esta la nag en pantalla vemos en el stack:

```
0012FC48  77D13C6B  RETURN to USER32.77D13C6B
0012FC4C  0058B8C0  RETURN to tute_.0058B8C0 from <JMP.&user32.WaitMessage>
0012FC50  0012FCA4  Pointer to next SEH record
0012FC54  0058B8DB  SE handler
0012FC58  0012FC78
0012FC5C  00622168  tute_.00622168
0012FC60  00D618BC
0012FC64  00D618BC
0012FC68  00000000
0012FC6C  00000000
0012FC70  01622168
0012FC74  00D618BC
0012FC78  0012FCF0
0012FC7C  0058B074  RETURN to tute_.0058B074 from tute_.0058B7EC
0012FC80  00BD0776
0012FC84  00000113
0012FC88  00000001
0012FC8C  00000000
0012FC90  01EBC947
0012FC94  0000021D
0012FC98  00000146
0012FC9C  00632D60  tute_.00632D60
0012FCA0  00587CC4  RETURN to tute_.00587CC4 from tute_.0058B058
0012FCA4  0012FCB0  Pointer to next SEH record
0012FCA8  00587D47  SE handler
```

pulsamos Enter sobre la línea resaltada y vemos el loop en el que está la nag a espera de que pulsemos algún botón de los tres que tiene:

```
00587CBD  >   8B03          MOV EAX,DWORD PTR DS:[EBX]
00587CBF  .   E8 94330000   CALL tute_.0058B058
00587CC4  .   8B03          MOV EAX,DWORD PTR DS:[EBX]
00587CC6  .   80B8 8C00000( CMP BYTE PTR DS:[EAX+8C],0
00587CCD  .v  74 0F         JE SHORT tute_.00587CDE
00587CCF  .   8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
00587CD2  .   C780 3402000( MOV DWORD PTR DS:[EAX+234],2
00587CDC  .v  EB 14         JMP SHORT tute_.00587CF2
00587CDE  >   8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
00587CE1  .   83B8 3402000( CMP DWORD PTR DS:[EAX+234],0
00587CE8  .v  74 08         JE SHORT tute_.00587CF2
00587CEA  .   8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
00587CED  .   E8 26FDFFFF   CALL tute_.00587A18
00587CF2  >   8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
00587CF5  .   8B80 3402000( MOV EAX,DWORD PTR DS:[EAX+234]
00587CFB  .   85C0          TEST EAX,EAX
00587CFD  .^  74 BE         JE SHORT tute_.00587CBD              ;fin_loop...
00587CFF  .   8945 F8       MOV DWORD PTR SS:[EBP-8],EAX
00587D02  .   6A 00         PUSH 0
00587D04  .   6A 00         PUSH 0
```

Ese es el bucle en el que está loopeando la ventana, dejamos correr Olly con F9, pulsamos OK y paramos en el BP, a partir de aquí traceamos superando los retornos hasta llegar a la zona que antes ya encontramos, salimos en :40636B intentaremos evitar la nag saltando en :40633C, ponemos un BP allí y reiniciamos Olly, cuando pare miramos el valor de [EDX+72D] en la ventana de dump:

```
0040633C  .   80BA 2D07000( CMP BYTE PTR DS:[EDX+72D],0          ;bonita comparacion...
00406343  .v  0F85 8000000( JNZ tute_.004063C9                   ;ojo a este salto...
00406349  .   8B4D FC       MOV ECX,DWORD PTR SS:[EBP-4]
0040634C  .   B2 01         MOV DL,1
```

Para el BP, miremos el valor de la variable:

```
Address   Hex dump                                          ASCII
00D6FBD1  00 00 01 00 00 00 00 2C C4 D8 00 00 00 00 00 00  ..0....,-ï......
00D6FBE1  00 01 00 88 FE D6 00 94 60 D7 00 00 00 00 00 0A  .0.-þÖ."`×......
00D6FBF1  00 00 00 0A 00 00 00 2C 01 00 00 96 00 00 00 00  .......,0..ü....
00D6FC01  01 00 00 88 5D D8 00 A0 BC D8 00 54 70 D8 00 C8  0..è]ï.á¼ï.Tpï.È
00D6FC11  73 D8 00 00 01 01 01 01 01 00 00 01 00 00 00 00  sï..00000..0....
```

El byte bajo está puesto a 0, lo cambiamos a 1 a ver que sucede, jeje…

```
Address   Hex dump                                          ASCII
00D6FBD1  01 00 01 00 00 00 00 2C C4 D8 00 00 00 00 00 00  ☺.☺.....,─ï......
00D6FBE1  00 01 00 88 FE D6 00 94 60 D7 00 00 00 00 00 0A  .☺.─þÖ."`×......
00D6FBF1  00 00 00 0A 00 00 00 2C 01 00 00 96 00 00 00 00  .......,☺..û....
00D6FC01  01 00 00 88 5D D8 00 A0 BC D8 00 54 70 D8 00 C8  ☺..ê]ï.á¼ï.Tpï.╚
00D6FC11  73 D8 00 00 01 01 01 01 01 00 00 01 00 00 00 00  sï..00000..☺....
```

Damos RUN y el programa arranca sin la nag y en modo licenciado xDD haremos el parchecito ya que estamos, cambiamos:

```
0040633C   80BA 2D070000> CMP BYTE PTR DS:[EDX+72D],0
00406343   0F85 80000000  JNZ tute_.004063C9

POR:
0040633C   C682 2D070000> MOV BYTE PTR DS:[EDX+72D],1
00406343   E9 81000000    JMP tute_.004063C9
00406348   90             NOP
```

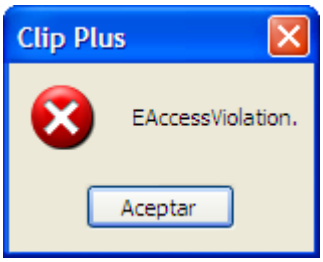
Salvamos los cambios como tute.exe, lo probamos y… nos da el mismo error AccesViolation de antes, necesitamos saber porqué no corre fuera de Olly, este error lo vamos a pillar como sea, se me ocurre a bote pronto dejar el programa en un bucle infinito en cualquier punto, salvar los cambios y ejecutarlo, si falla sabemos que el error está antes y si no es que está depués, dejo el loop aquí:

```
00405D2A  .  8B08          MOV ECX,DWORD PTR DS:[EAX]
00405D2C  .  FF51 3C       CALL DWORD PTR DS:[ECX+3C]
00405D2F  .  B2 01         MOV DL,1
00405D31  .  A1 D4595700   MOV EAX,DWORD PTR DS:[5759D4]
00405D36  .  E8 F1681700   CALL clipplus.0057C62C
00405D3B  -  EB FE         JMP SHORT clipplus.00405D3B
00405D3D  .  8B45 FC       MOV EAX,DWORD PTR SS:[EBP-4]
00405D40  .  BA 2EA36100   MOV EDX,clipplus.0061A32E      ASCII "X_GLYPH_SOUND"
```

Salvo los cambios, lo ejecuto y se queda en el loop, no falla, luego el error está después de esta dirección, quito el loop y lo creo en otro punto más adelante:

```
00406353  .  E8 64600100   CALL clipplus.0041C3BC
00406358  .  8BD8          MOV EBX,EAX
0040635A  .  A1 540C6300   MOV EAX,DWORD PTR DS:[630C54]
0040635F  .  8918          MOV DWORD PTR DS:[EAX],EBX
00406361  .  8BC3          MOV EAX,EBX
00406363  -  EB FE         JMP SHORT clipplus.00406363
00406365  .  FF92 D8000000 CALL DWORD PTR DS:[EDX+D8]
0040636B  .  8B0D 540C6300 MOV ECX,DWORD PTR DS:[630C54]   clipplus._RegNoteForm
00406371  .  8B19          MOV EBX,DWORD PTR DS:[ECX]
```

Salvo los cambios y falla…



Entonces sabemos que el error está entre :405D3B y :406363, ahora es cuestión de hacer un cerco hasta pillar el momento justo del error, el siguiente punto será una direción intermedia como por ejemplo :405F6D y aquí no falla, o sea que está después, el cerco se acorta, ahora nos queda entre:

405F6D - 406363

Buscamos una dirección intermedia para crear el loop, por ejemplo :40611B, lo ejecutamos y tampoco falla ;-) el cerco se acorta señores, ahora queda el intervalo:g

40611B - 406363

Entre estas direcciones está el error, creamos el loop en :4062AF tampoco falla, queda:

4062AF - 406363

Elegimos ahora por ejemplo :40631F y tampoco falla, ya queda muuuuuy poco para dar con él, nos queda este pequeño margen, jeje…

40631F - 406363

Elegimos esta vez :406339 y FALLA, jeje.. el error está pues entre estas direcciones:

40631F - 406339

Seguimos así hasta dar con él, no lo anoto todo porque se haría interminable, no he de decir supongo que para cada loop creado necesitamos un archivo virgen "sin cambios" para que no afecten al nuevo que creamos xDD

Al final lo localizamos en la dirección :406334 donde está este CALL:

00406334 E8 53780000 CALL clipplus.0040DB8C

Este es el culpable de que no funcione, intentaremos evitarlo, entramos dentro y falla el siguiente subcall:

0040DBA4 . E8 CBB91C00 CALL clipplus.005D9574

este es el responsible de todo, si lo miramos con lupa vemos que realmente no hace mucho pero que no veas como molesta ;-)

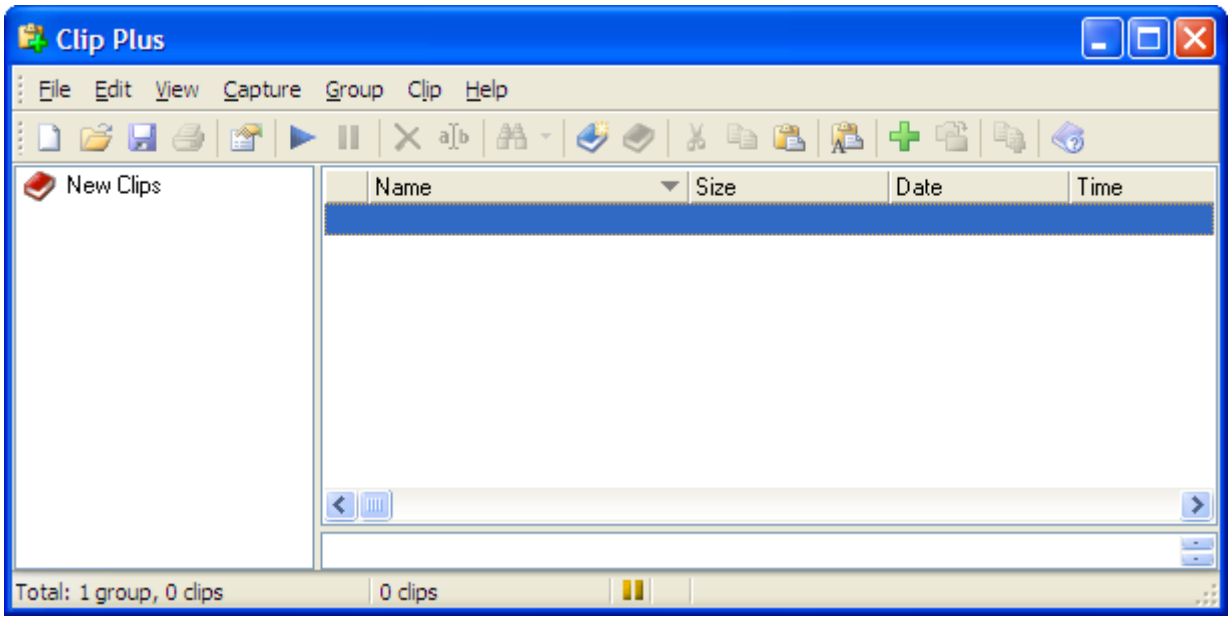
El cambio es evidente probamos lo más fácil siempre como primera opcion, esto dice el teorema fundamental de la lógica verdad ¿? Pues eso haremos, el cambio:

```
0040DB93  .  8BD8          MOV EBX,EAX
0040DB95  .  B8 14ED6100   MOV EAX,tute_.0061ED14
0040DB9A  .  E8 79861D00   CALL tute_.005E6218
0040DB9F  .  A1 5C9A6100   MOV EAX,DWORD PTR DS:[619A5C]
0040DBA4  .  E8 CBB91C00   CALL tute_.005D9574
0040DBA9  .  85C0          TEST EAX,EAX
0040DBAB  .v 76 4E         JBE SHORT tute_.0040DBFB
0040DBAD  .v E9 47000000   JMP tute_.0040DBF9
0040DBB2     C7            DB C7
0040DBB3     C5            DB C5
0040DBB4     E3            DB E3
0040DBB5     7D            DB 7D                    CHAR '}'
```

Que cambiaremos por esto otro:

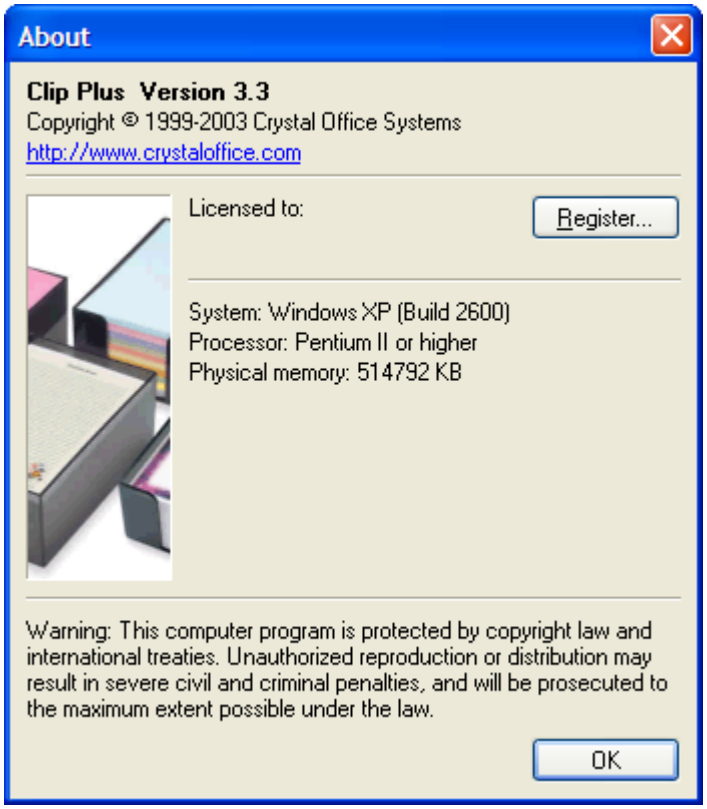
```
0040DB93     8BD8          MOV EBX,EAX
0040DB95  .  B8 14ED6100   MOV EAX,clipplus.0061ED14
0040DB9A  .  E8 79861D00   CALL clipplus.005E6218
0040DB9F     A1 5C9A6100   MOV EAX,DWORD PTR DS:[619A5C]
0040DBA4     90            NOP
0040DBA5     90            NOP
0040DBA6     90            NOP
0040DBA7     90            NOP
0040DBA8     90            NOP
0040DBA9  .  85C0          TEST EAX,EAX
0040DBAB   v 76 4E         JBE SHORT clipplus.0040DBFB
0040DBAD  .v E9 47000000   JMP clipplus.0040DBF9
0040DBB2     C7            DB C7
0040DBB3     C5            DB C5
0040DBB4     E3            DB E3
0040DBB5     7D            DB 7D                    CHAR '}'
0040DBB6     20            DB 20                    CHAR ' '
```

lo que hacemos es evitar que ejecute el CALL para que no nos dé más problemas de los que ya por sí tenemos, salvamos los cambios como tute_ok.exe, lo probamos y…



jejeje… RULAAAAAAAAAAaaa, al final funciona, ha costado pero funciona que es lo importante, bouuuuuuuffff..!!

Y bien si miramos en el About ahora:



Dice "Licensed to:" aunque no pone ningún nombre ya que no nos hemos registrado y creo que tenía la opción opr ahí pero hemos ido por otr caminito que nos llevó al mismo lado, a mi por lo menos ya me vale esta opción ;-)

Espero que hayáis pasado un buen rato con el tute, a mi me ha encantado escribirlo, lo he hecho con las mejores ideas que he podido, si algo no quedío bastante claro ya sabéis me escribís al privado y lo comentamos, esta es mi dirección:

syxe00@yahoo.es

hastala próxima, saludos a todo el TEAM xDD